LA CITÉ

Les Transactions en SQL server

Plan du cours

- **Objectifs**
- **Création d’une transaction**
- **Validation (commit)**
- **Annulation (rollback)**
- **Points de sauvegarde (savepoint)**

Objectifs

Ce cours a pour but de vous apprendre à :

- Créer et contrôler des transactions en SQL Server ;
- Comprendre la différence entre COMMIT et ROLLBACK ;
- Utiliser des points de sauvegarde (SAVE TRANSACTION) pour mieux gérer les erreurs ;
- Garantir l'intégrité et la cohérence des données dans tes traitements SQL.

Les transactions en SQL

- **Définition et Utilité des transactions**
- Une transaction est un ensemble d'instructions SQL exécutées comme une seule unité logique de travail. Cette une unité logique de travail regroupe plusieurs opérations SQL (INSERT, UPDATE, DELETE…) afin qu'elles soient exécutées comme un tout indivisible.

➢ Soit toutes les instructions réussissent, soit aucune n'est appliquée.

- Les transactions sont fondamentales pour garantir la fiabilité des bases de données.
- On utilise les transactions chaque fois qu'on veut garantir la cohérence des données sur un ensemble d'opérations dépendantes les unes des autres.

➢ **Quand l'utiliser**? Dès qu' il y a plusieurs opérations dépendantes entre elles

Les transactions en SQL

• Définition et Utilité des transactions

- **Problème en l absence de transaction:**

✓ **Exemple simple sans transaction :**

```
INSERT INTO Commandes VALUES (1, 'Produit A', 100); --instruction 1
UPDATE Stock SET Quantite = Quantite - 1 WHERE IdProduit = 1; --instruction  2
```

➢ **Si la deuxième instruction Update échoue, la première Insert reste quand même enregistré → incohérence possible puisque on va ajouter un produit à une commande sans mettre à jour le stock de ce produit.**

➢ **D où l importance des transactions qui permettent d exécuter l ensemble des instructions SQL comme un seul bloc (dans l exemple soit le insert et le update réussissent soit les deux échouent) ce qui permet d éviter les incohérences des données**

Les transactions en SQL

- **Cas typiques d utilisation des transactions**

Situation	Exemple concret
Opérations dépendantes entre plusieurs tables	Insertion d'un client et de sa commande : les deux doivent exister ensemble.
Mise à jour multiple liée	Débiter un compte et créditer un autre (transfert bancaire).
Suppression en cascade manuelle	Supprimer un client et toutes ses commandes associées.
Traitement complexe dans une procédure	Calculer des totaux, insérer des lignes, puis valider tout le bloc.
Restauration partielle	Revenir à un point précis sans tout annuler (SAVE TRANSACTION).

Les transactions en SQL

• Les avantages des transactions

- Les transactions sont fondamentales pour garantir la fiabilité des bases de données.

❖ **Atomicité**

- Toutes les instructions s'exécutent entièrement ou pas du tout.

➢ **Empêche les modifications partielles.**

❖ **Cohérence**

- Les contraintes d'intégrité (clés primaires, étrangères, règles métiers) restent valides.

➢ **La base reste toujours dans un état correct.**

❖ **Isolation**

- Les transactions concurrentes ne se perturbent pas.

➢ **Une transaction en cours n'interfère pas avec une autre.**

❖ **Durabilité**

- Une fois validée (COMMIT), la transaction est permanente, même après un redémarrage.

➢ **Les données sont garanties.**

Les transactions en SQL

• Syntaxe de création de transaction

Une transaction commence par la commande :Begin transaction et se termine par commit ou rollback transaction

BEGIN TRANSACTION

instructions SQL -- Tout ce qui suit fait partie de la transaction jusqu'à ce qu'elle soit :**validée** (COMMIT), ou **annulée** (ROLLBACK).

COMMIT TRANSACTION - - valider la transaction

-- ou

ROLLBACK TRANSACTION - - annuler la transaction

Les transactions en SQL

• Exemple de création et de validation de transaction

Commencer par créer la base de données COMMERCETRANSACTION avec les tables Clients et Commande en utilisant le fichier commercetransaction.sql

SQLQuery1.sql - M...CCAD\WOuesl (52))* | base gestionclients.sql - non con

```sql
CREATE DATABASE COMMERCETRANSACTION;
USE COMMERCETRANSACTION;
CREATE TABLE Clients (
        IdClient INT IDENTITY PRIMARY KEY,
        Nom NVARCHAR(50)
);

CREATE TABLE Commandes (
        IdCommande INT IDENTITY PRIMARY KEY,
        IdClient INT,
        Montant DECIMAL(10,2)
);
```

100 %

Messages

```
Commandes réussies.

Heure de fin : 2025-11-09T11:06:05.4282866-05:00
```

Les transactions en SQL

• Exemple de création et de validation de transaction

Exemple concret : Ajout d'un client et de sa commande

on veut créer utilise une transaction pour **ajouter un client et sa commande ensemble** :

```sql
--on veut créer un client et sa commande ensemble au meme temps
BEGIN TRANSACTION;-- creation de la transaction

INSERT INTO Clients (Nom)-- ajout d un client
VALUES ('Alice');

INSERT INTO Commandes (IdClient, Montant)-- ajout de la commande pour ce client
VALUES (SCOPE_IDENTITY(), 200.00);--pour récupérer l'identifiant du client que on vient d'insérer avec un identifiant de type identity, on ne peut pas le deviner.
                                  -- la fonction intégrée de sql SCOPE_IDENTITY() nous donne exactement cette valeur.

COMMIT TRANSACTION;--valide toutes les opérations effectuées depuis le BEGIN TRANSACTION.Une fois le COMMIT exécuté :
                   --Les modifications deviennent permanentes. Elles ne peuvent plus être annulées.
```

100 %

Messages

```
(1 ligne affectée)

(1 ligne affectée)

Heure de fin : 2025-11-09T11:19:46.3192410-05:00
```

- **Si les deux requêtes réussissent → les changements sont enregistrés.**
- **Si la deuxième échoue → tout est annulé (rien n'est ajouté).**

• Exemple de création et de validation de transaction

Exemple concret : Ajout d'un client et de sa commande

on veut créer utilise une transaction pour **ajouter un client et sa commande ensemble** :

Vérification de l insertion dans Clients et dans Commandes

```sql
-- vérification de l insertion dans Clients
SELECT * FROM Clients;
```

100 %

Résultats | Messages

	IdClient	Nom
1	1	Alice

```sql
-- vérification de l insertion dans Commandes
SELECT * FROM Commandes;
```

100 %

Résultats | Messages

	IdCommande	IdClient	Montant
1	1	1	200.00

Les transactions en SQL

- **Exemple de création et de validation de transaction avec l affichage d un message**

Exemple concret : Modification du nom du client 1

on veut créer une transaction qui permet de modifier le nom du **client 1et d afficher un message à l utilisateur** :

```sql
-- on veut créer une transaction qui permet changer le nom du client numéro 1 et qui affiche un message à l utilisateur
BEGIN TRANSACTION;
UPDATE Clients
SET Nom = 'Alice Dupont'
WHERE IdClient = 1;
COMMIT TRANSACTION;
PRINT 'Modification validée avec succès.';
```

100 %

Messages

```
(1 ligne affectée)
Modification validée avec succès.

Heure de fin : 2025-11-09T11:27:51.6349783-05:00
```

- **Le COMMIT finalise la transaction → le changement du nom est définitivement enregistré.**

Les transactions en SQL

- **Exemple de création et de validation de transaction avec l affichage d un message**

Exemple concret : Modification du nom du client 1

on veut créer une transaction qui permet de modifier le nom du **client 1et d afficher un message à l utilisateur**

Vérification de la mise à jour du nom dans la table Clients

```
-- vérification de la mise à jour du nom du client 1
SELECT * FROM Clients WHERE IdClient=1;
```

100 %

Résultats | Messages

	IdClient	Nom
1	1	Alice Dupont

Les transactions en SQL

- **Retour en arriére Rollback pour annuler la transaction**
- Le **moment du COMMIT** et du **ROLLBACK** dépend du moment où on est sûr que tout est correct.
  on ne peut pas “commettre” d’abord puis faire un ROLLBACK ensuite.
  Une fois le COMMIT TRANSACTION exécuté, **les changements sont définitivement enregistrés dans la base** , on ne peut plus les annuler avec **ROLLBACK.**
- SQL Server gère les transactions comme des blocs atomiques :
- Tant que la transaction **n’est pas validée (COMMIT)**, tout peut être **annulé (ROLLBACK)**.
- Mais **après le COMMIT**, la transaction est **finalisée** : SQL Server a écrit les changements sur disque et libéré les verrous.
- Donc la logique doit être :

1. **On suppose que tout est faux ou incertain.**
2. On **vérifie les conditions**.
3. Si tout est bon → **COMMIT TRANSACTION**.
4. Sinon → **ROLLBACK TRANSACTION.**

Les transactions en SQL

- **Exemple de Retour en arriére Rollback pour annuler la transaction si la condition n est pas vérifiée**
- **Exemple conret: insertion d une commande avec rollback et commit**

On veut créer une transaction qui permet d insérer une commande pour le client 1, si le montant est négatif la transaction sera annulée (rollback) sinon la transaction commande sera validée(commit); montant positif

```sql
-- creation de transaction qui permet d inserer une commande pour le client 1,
--si le montant est négatif la transaction sera annulée (rollback) sinon la transaction commande sera validée(commit)
--tester avec un montant positif
BEGIN TRANSACTION;
DECLARE @IdClient INT = 1;
DECLARE @Montant DECIMAL(10,2) = 400.00;
-- On insère une commande
INSERT INTO Commandes (IdClient, Montant)
VALUES (@IdClient, @Montant);
-- Supposons qu'on "pense" que tout est bon ici : Mais tant qu'on n'a pas fait COMMIT, on peut encore annuler !
-- Vérifions la condition après coup :
IF @Montant < 0
BEGIN
    ROLLBACK TRANSACTION; --  Annule l'insertion
    PRINT 'Erreur : montant négatif, on annule.';
END
ELSE
BEGIN
    COMMIT TRANSACTION; --  Confirme définitivement l'insertion
    PRINT 'Transaction validée.';
END
```

.00 %

Résultats | Messages

➢ **Le montant de la commande est positif donc c est le COMMIT qui est exécuté**

Les transactions en SQL

- **Exemple de Retour en arriére Rollback pour annuler la transaction si la condition n est pas vérifiée**
- **Exemple conret: insertion d une commande avec rollback et commit**

On veut créer une transaction qui permet d insérer une commande pour le client 1, si le montant est négatif la transaction sera annulée (rollback) sinon la transaction commande sera validée(commit); montant négatif

```sql
-- creation de transaction qui permet d inserer une commande pour le client 1,
--si le montant est négatif la transaction sera annulée (rollback) sinon la transaction commande sera validée(commit)
--tester avec un montant négatif
BEGIN TRANSACTION;
DECLARE @IdClient INT = 1;
DECLARE @Montant DECIMAL(10,2) = -300.00;
-- On insère une commande
INSERT INTO Commandes (IdClient, Montant)
VALUES (@IdClient, @Montant);
-- Supposons qu'on "pense" que tout est bon ici : Mais tant qu'on n'a pas fait COMMIT, on peut encore annuler !
-- Vérifions la condition après coup :
IF @Montant < 0
BEGIN
    ROLLBACK TRANSACTION; --  Annule l'insertion
    PRINT 'Erreur : montant négatif, on annule.';
END
ELSE
BEGIN
    COMMIT TRANSACTION; --  Confirme définitivement l'insertion
    PRINT 'Transaction validée.';
END
```

00 %

Messages

```
(1 ligne affectée)
Erreur : montant négatif, on annule.

Heure de fin : 2025-11-10T12:59:03.9107241-05:00
```

➢ **Le montant de la commande est négatif donc c est le ROLLBACK qui est exécuté, du coup l insertion est annulée**

Les transactions en SQL

Exemple concret : insertion d une commande avec rollback et commit

On veut créer une transaction qui permet d insérer une commande pour le client 1, si le montant est négatif la transaction sera annulée (rollback) sinon la transaction commande sera validée(commit);

Vérification de l insertion dans Commandes

Select * from Commandes

```
-- verification de l insertion dans la table commande
SELECT * FROM Commandes;
```

100 %

Résultats | Messages

	IdCommande	IdClient	Montant
1	1	1	200.00
2	4	1	400.00

La nouvelle commande avec le montant positif (400) a été bien inseré dans la table Commandes

Les transactions en SQL

- **Exemple de création et de validation ou d annulation de transaction**
- **On veut créer une transaction qui permet de modifier un client si il existe et d annuler la transaction si il n existe pas ou bien si la modification correspond à l état initial ; changer le nom par le même nom par exemple**
- **on va travailler sur l exemple où le client à modifier n existe pas IdClient=999**

```sql
-- creation de transaction qui permet de modifier un client si il existe et d annuler la transaction si il n existe pas
BEGIN TRANSACTION;
-- Étape 1 : Vérification de l'existence du client avant la mise à jour
IF NOT EXISTS (SELECT 1 FROM Clients WHERE IdClient = 999)--Si cette ligne existe, alors la requête renvoie quelque chose (le nombre 1).
                                                          --Si elle n'existe pas, la requête ne renvoie rien.
                                                          --Donc ici, le but n'est pas de récupérer une valeur utile, mais juste de tester l'existence d'une ligne.

BEGIN
    PRINT 'Erreur : le client avec IdClient = 999 n'existe pas.';
    ROLLBACK TRANSACTION;  -- On annule car le client n'existe pas
END
ELSE
BEGIN
    -- Étape 2 : Mise à jour du client s'il existe
    UPDATE Clients
    SET Nom = 'Bob'
    WHERE IdClient = 999;

    IF @@ROWCOUNT > 0  --  @@ROWCOUNT retourne uniquement le nombre de lignes réellement modifiées par la dernière commande exécutée.
    --     Il ne compte PAS le nombre total de lignes de la table.Exemple :Si IdClient = 999 n'existe pas → @@ROWCOUNT = 0
    --     Si une seule ligne est mise à jour → @@ROWCOUNT = 1
    BEGIN
        COMMIT TRANSACTION;  -- Validation de la modification
        PRINT 'Mise à jour effectuée et validée avec succès.';
    END
    ELSE
    BEGIN
        PRINT 'Aucune ligne mise à jour - rollback par sécurité.'; -- si dans notre update la valeur du champ est restée identique ('Alice Dupont' -> 'Alice Dupont')
                                                                  -- donc SQL Server ne considère pas qu'une ligne a été modifiée,Donc @@ROWCOUNT = 0
                                                                  -- donc on est dans la partie else de   IF @@ROWCOUNT > 0
        ROLLBACK TRANSACTION;  -- Annule si rien n'a été modifié, ca sera exécuter si le nom reste le meme dans l instruction update
    END
END
```

100 %

Messages

```
Erreur : le client avec IdClient = 999 n'existe pas.
```

100 %

Les transactions en SQL

- **Exemple de création et de validation ou d annulation de transaction**
- **On veut créer une transaction qui permet de modifier un client si il existe et d annuler la transaction si il n existe pas ou bien si la modification correspond à l état initial ; changer le nom par le même nom par exemple**
- **on va travailler sur l exemple où le client à modifier existe IdClient=1**

```sql
--on va travailler sur l exemple où le client à modifier  existe
BEGIN TRANSACTION;
-- Étape 1 : Vérification de l'existence du client avant la mise à jour
IF NOT EXISTS (SELECT 1 FROM Clients WHERE IdClient = 1)--Si cette ligne existe, alors la requête renvoie quelque chose (le nombre 1).
                                                        --Si elle n'existe pas, la requête ne renvoie rien.
                                                        --Donc ici, le but n'est pas de récupérer une valeur utile, mais juste de tester l'existence d'une ligne.
BEGIN
    PRINT 'Erreur : le client avec IdClient = 999 n'existe pas.';
    ROLLBACK TRANSACTION;  -- On annule car le client n'existe pas
END
ELSE
BEGIN
    -- Étape 2 : Mise à jour du client s'il existe
    UPDATE Clients
    SET Nom = 'Bob'
    WHERE IdClient = 1;

    IF @@ROWCOUNT > 0  --  @@ROWCOUNT retourne uniquement le nombre de lignes réellement modifiées par la dernière commande exécutée.
    --      Il ne compte PAS le nombre total de lignes de la table.Exemple :Si IdClient = 999 n'existe pas → @@ROWCOUNT = 0
    --      Si une seule ligne est mise à jour → @@ROWCOUNT = 1
    BEGIN
        COMMIT TRANSACTION;  -- Validation de la modification
        PRINT 'Mise à jour effectuée et validée avec succès.';
    END
    ELSE
    BEGIN
        PRINT 'Aucune ligne mise à jour — rollback par sécurité.'; -- si dans notre update la valeur du champ est restée identique ('Alice Dupont' -> 'Alice Dupont')
                                                                  -- donc SQL Server ne considère pas qu'une ligne a été modifiée,Donc @@ROWCOUNT = 0
                                                                  -- donc on est dans la partie else de   IF @@ROWCOUNT > 0
        ROLLBACK TRANSACTION;  -- Annule si rien n'a été modifié, ca sera exécuter si le nom reste le meme dans l instruction update
    END
END
```

100 %

Messages

```
(1 ligne affectée)
Mise à jour effectuée et validée avec succès.
```

Les transactions en SQL

- **Exemple de création et de validation ou d annulation de transaction**
- **On veut créer une transaction qui permet de modifier un client si il existe et d annuler la transaction si il n existe pas ou bien si la modification correspond à l état initial ; changer le nom par le même nom par exemple**
- **on va travailler sur l exemple où le client à modifier existe IdClient=1**
- **Vérification de la mise à jour du nom du client 1**

```
END
-- verification de la mise à jour du nom du client numero 1
SELECT * FROM Clients WHERE IdClient=1;
```

100 %

Résultats | Messages

	IdClient	Nom
1	1	Bob

Les transactions en SQL

- **Exemple de création et de validation ou d annulation de transaction**
- **On veut créer une transaction qui permet de modifier un client si il existe et d annuler la transaction si il n existe pas ou bien si la modification correspond à l état initial ; changer le nom par le même nom par exemple**
- **on va travailler sur l exemple où la mise à jour du nom du client à modifier est la même que son ancien nom**

commercetransacti...CCAD\WOuesl (60))* | base gestionclients.sql - non connecté

```sql
--on va travailler sur 1 exemple ou le client à modifier existe
BEGIN TRANSACTION;
-- Étape 1 : Vérification de l'existence du client avant la mise à jour
IF NOT EXISTS (SELECT 1 FROM Clients WHERE IdClient = 1)--Si cette ligne existe, alors la requête renvoie quelque chose (le nombre 1).
                                                        --Si elle n'existe pas, la requête ne renvoie rien.
                                                        --Donc ici, le but n'est pas de récupérer une valeur utile, mais juste de tester l'existence d'une ligne.
BEGIN
    PRINT 'Erreur : le client avec IdClient = 999 n'existe pas.';
    ROLLBACK TRANSACTION;  -- On annule car le client n'existe pas
END
ELSE
BEGIN
    -- Étape 2 : Mise à jour du client s'il existe
    UPDATE Clients
    SET Nom = 'Bob'
    WHERE IdClient = 1;

    IF @@ROWCOUNT > 0  --  @@ROWCOUNT retourne uniquement le nombre de lignes réellement modifiées par la dernière commande exécutée.
    --      Il ne compte PAS le nombre total de lignes de la table.Exemple :Si IdClient = 999 n'existe pas → @@ROWCOUNT = 0
    --      Si une seule ligne est mise à jour → @@ROWCOUNT = 1
    BEGIN
        COMMIT TRANSACTION;  -- Validation de la modification
        PRINT 'Mise à jour effectuée et validée avec succès.';
    END
    ELSE
    BEGIN
        PRINT 'Aucune ligne mise à jour – rollback par sécurité.'; -- si dans notre update la valeur du champ est restée identique ('Bob' -> 'Alice Bob')
                                                                  -- donc SQL Server ne considère pas qu'une ligne a été modifiée,Donc @@ROWCOUNT = 0
                                                                  -- donc on est dans la pertie else de   IF @@ROWCOUNT > 0
        ROLLBACK TRANSACTION;  -- Annule si rien n'a été modifié, ca sera exécuter si le nom reste le meme dans 1 instruction update
    END
END
```

100 %

Messages

```
(1 ligne affectée)
Mise à jour effectuée et validée avec succès.
```

100 %

Exécution de requête réussie. | MOB-4MPVHL3 (15.0 RTM) | LCCAD\WOuesl (60) | COMMERCETRANSACTION | 00:00:00 | 0 lignes

Les transactions en SQL

- **Exemple de création et de validation ou d annulation de transaction**
- **On veut créer une transaction qui permet de modifier un client si il existe et d annuler la transaction si il n existe pas**
- **on va travailler sur l exemple où la mise à jour du nom du client à modifier est la même que son ancien nom**

```sql
-- transaction pour mettre à jour le nom du client si existe
--on ne traite pas le cas où le nom modifié est le meme que l ancien nom
BEGIN TRANSACTION;

UPDATE Clients
SET Nom = 'Bob'
WHERE IdClient = 1; -- Ce client existe pas

-- Vérification
IF @@ROWCOUNT = 0
BEGIN
    PRINT 'Erreur : aucun client trouvé, retour en arrière.';
    ROLLBACK TRANSACTION;
END
ELSE
BEGIN
    COMMIT TRANSACTION;
    PRINT 'Mise à jour effectuée et validée avec succès.';
END
```

100 %

Messages

```
(1 ligne affectée)
Mise à jour effectuée et validée avec succès.
```

100 %

- **La transaction a exécuté le COMMIT puisque le cas de mise à jour du nom par l ancien nom n a pas été traité dans cette transaction**

Les transactions en SQL

- **Points de sauvegarde (SAVE TRANSACTION)**
- **Les points de sauvegarde** (*savepoint*) **permettent de revenir partiellement en arrière dans une transaction sans tout annuler.**
- Un **point de sauvegarde** est un **marqueur intermédiaire** à l'intérieur d'une transaction, qui te permet de **revenir à une étape précise** sans annuler toute la transaction.
- **Syntaxe de point de sauvegarde**

SAVE TRANSACTION NomPoint;

Puis on peut revenir à ce point avec :

ROLLBACK TRANSACTION NomPoint;

Les transactions en SQL

- **Points de sauvegarde (SAVE TRANSACTION)**
- **Pourquoi utiliser un point de sauvegarde ?**
- Quand ta transaction contient plusieurs étapes successives (plusieurs requêtes), on peut vouloir **annuler seulement une partie**, sans perdre tout le reste.
- **Exemple :**

✓ On insère un client, et on crée un point de sauvegarde

✓ puis on tente d'ajouter sa commande,

✓ Si seule l'ajout de la commande échoue.

➢ Grâce à un point de sauvegarde, on peut **revenir avant la commande** tout en **gardant le client**.

Les transactions en SQL

- **Points de sauvegarde (SAVE TRANSACTION)**
- **Pourquoi utiliser un point de sauvegarde ?**
- Quand ta transaction contient plusieurs étapes successives (plusieurs requêtes), on peut vouloir **annuler seulement une partie**, sans perdre tout le reste.
- **Exemple : créer une transaction d ajout de client et de commande avec un point de sauvegarde**

```sql
--creation de transaction pour inserer un nouveau client et une commande pour un client avec des points de sauvegardes
BEGIN TRANSACTION;
-- Étape 1 : insertion d'un client
INSERT INTO Clients (Nom) VALUES ('Sami');
PRINT 'Client ajouté.';
--  Création d'un point de sauvegarde
SAVE TRANSACTION PointClient;
-- Étape 2 : tentative d'ajouter une commande
-- (ici on se trompe volontairement : IdClient inexistant)
INSERT INTO Commandes (IdClient, Montant) VALUES (999, 300.00);
-- Vérification d'erreur
IF @@ERROR <> 0--@@ERROR est une variable système dans SQL Server qui contient le code d'erreur retourné par la dernière instruction T-SQL exécutée.
                --Si l'instruction s'est bien exécutée, @@ERROR vaut 0.
                --Si une erreur s'est produite (ex : contrainte violée, clé étrangère manquante, etc.), @@ERROR contient un numéro d'erreur spécifique (≠ 0).
BEGIN
    PRINT 'Erreur détectée, retour au point de sauvegarde.';
    ROLLBACK TRANSACTION PointClient;  -- revient à l'état juste après le client
END
-- Étape 3 : on valide ce qui reste
COMMIT TRANSACTION;
PRINT 'Transaction validée.';
```

100 %

Messages

```
(1 ligne affectée)
Client ajouté.
```

Les transactions en SQL

- **Points de sauvegarde (SAVE TRANSACTION)**

- Le client Sami est **ajouté dans la table Clients.**
- Un **point de sauvegarde** PointClient est créé juste **après cette insertion**.
- Une commande est ensuite tentée pour **un IdClient = 999, qui n'existe pas**.
- **L'erreur** déclenche un **rollback partiel** vers **PointClient.**
- Cela **annule uniquement la commande**, mais **le client reste enregistré**.
- Enfin, **COMMIT TRANSACTION valide l'état actuel**

➢ **le client Sami est conservé,**

➢ **la commande erronée n'est pas insérée.**

Les transactions en SQL

- **Points de sauvegarde (SAVE TRANSACTION)**
- **Exemple : créer une transaction d ajout de deux clients avec un point de sauvegarde**

```sql
--creation d une transaction qui ajoute un client et execute un point de sauvegarde
--puis tente d ajouter un autre client mais annule par la suite cette opération
BEGIN TRANSACTION;
--ajout du client
INSERT INTO Clients (Nom)
VALUES ('Claire');
--sauvegarder l operation
SAVE TRANSACTION Point1;  -- Premier point de sauvegarde
--tentative d un deuxieme ajout d un client
INSERT INTO Clients (Nom)
VALUES ('David');
-- Supposons qu'on veuille annuler seulement le dernier ajout mais on garde le premier ajout:
ROLLBACK TRANSACTION Point1;--annule tout le code aprés save transaction et s arrete à ce niveau

COMMIT TRANSACTION;--valide le code avant SAVE TRANSACTION Point1;
```

100 %

Messages

```
(1 ligne affectée)

(1 ligne affectée)

Heure de fin : 2025-11-10T15:11:10.7540362-05:00
```

Les transactions en SQL

- **Points de sauvegarde (SAVE TRANSACTION)**
- **Exemple : créer une transaction d ajout de client et de commande avec un point de sauvegarde**

Vérification de l insertion du client Claire et non pas David

Select * from Clients

```
-- verification de la premiere insertion du client Claire
SELECT * FROM Clients;-- que claire est inseré, David nà pas été inseré
```

100 %

Résultats | Messages

	IdClient	Nom
1	1	Bob
2	2	Sami
3	3	Claire

- Claire est **enregistrée** (**avant le point de sauvegarde**)
- David est **annulé** (**après le point de sauvegarde**)

Les transactions en SQL

- **La gestion des erreurs dans les transactions**
- La **gestion des erreurs dans les transactions** permet de garantir **l'intégrité des données lorsqu'une opération échoue**.

- En SQL Server, elle consiste à **détecter les erreurs** après chaque instruction et à **annuler la transaction** si nécessaire à l'aide de **ROLLBACK.**

- L'objectif est **d'éviter qu'une partie seulement** des modifications soit **enregistrée.**

- On peut utiliser la variable système **@@ERROR** pour vérifier si une erreur s'est produite après une commande, ou bien le bloc **TRY...CATCH**, plus moderne et lisible, pour centraliser la détection et le traitement des erreurs.

- Ainsi, **seules les opérations exécutées sans erreur** sont validées par un **COMMIT**, assurant la **cohérence** de la base de données.

Les transactions en SQL

- La gestion des erreurs dans les transactions
- Exemple : créer une transaction d ajout de client et de sa commande simultanément avec une gestion d erreur try catch

```sql
-- transaction avec gestion d erreur try catch pour l ajout d un client et sa commande simultanément
BEGIN TRY
    BEGIN TRANSACTION;

    INSERT INTO Clients (Nom) VALUES ('Emma');
    INSERT INTO Commandes (IdClient, Montant) VALUES (SCOPE_IDENTITY(), 350.00);

    COMMIT TRANSACTION;
    PRINT 'Transaction validée avec succès.';
END TRY
BEGIN CATCH
    PRINT 'Erreur détectée, annulation des opérations.';
    ROLLBACK TRANSACTION;
END CATCH;
```

100 %

Messages

```
(1 ligne affectée)

(1 ligne affectée)
Transaction validée avec succès.
```

Les transactions en SQL

- La gestion des erreurs dans les transactions
- Exemple : créer une transaction d ajout de client et de sa commande simultanément avec une gestion d erreur try catch

Vérification de l insertion du client et de sa commande

Select * from Clients;

Select * from Commandes;

```
--verification de l insertion dans clients et dans commandes
SELECT * FROM Clients;
SELECT * FROM Commandes;
```

100 %

Résultats | Messages

	IdClient	Nom
1	1	Bob
2	2	Sami
3	3	Claire
4	5	Emma

```
--verification de l insertion dans clients et dans commandes
SELECT * FROM Clients;
SELECT * FROM Commandes;
```

100 %

Résultats | Messages

	IdCommande	IdClient	Montant
1	1	1	200.00
2	4	1	400.00
3	5	999	300.00
4	6	5	350.00